DATA STRUCTURES :TECHNICAL PRESENTATION

ASSESSMENT TOOL 2

SUBECT CODE:CSA0308

Guided by:Dr.Rajaram

Done by:G Rupalakshmi
Reg.No:1911260197

SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

ASSESSMENT TOOL 2 - Technical Presentation

COURSE NAME:Data StructuresCOURSE CODE: CSA03

Register Number	1911260197
Name	G RUPALAKSHMI

Assessment Tool Weightage: 30%

COURSE OUTCOME COVERED

CO1: *Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL3, BL4)*

Questions:4**EACH QUESTIONS CARRIES:** 25 MARKS**Total Marks:**100**Duration:** ONE HOUR

S. No.	List of Questions
1	A student management system needs to store the marks of 1000 students efficiently. Apply the concepts of primitive and non-primitive data structures to select an appropriate structure for storing and managing the data and prepare a technical presentation justifying your choice based on the efficiency of data structure.
2	Choose a suitable linear or non-linear data structure for a navigation system that stores city connections and finds shortest routes between locations. Prepare a technical presentation explaining why it is appropriate.
3	A web browser supports navigating backward through previously visited pages. Recommend a suitable ADT and prepare a technical presentation explaining how it improves performance efficiency.
4	A digital library allows searching books using ISBN numbers quickly among 50,000 records. Select the most suitable data structure and Prepare a technical presentation justifying using time complexity comparison.

Code of Conduct:

I G RUPALAKSHMI(1911260197) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:G RUPALAKSHMI

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Problem: Student Grade Repository

High-Capacity Record Storage

Store performance metrics and academic marks for exactly 1,000 students efficiently.

Direct Access Requirements

System requires deterministic index lookup times to prevent processing bottlenecks during bulk queries.

Data Structure Evaluation

Evaluating trade-offs between atomic primitive types and complex non-primitive memory models.

ASSESSMENT MAPPING

"Designs an optimal contiguous memory layout for standardized grading registers."

Directly answers Question 1 specifications.

Primitive vs. Non-Primitive Categorization

Primitive Structures

Basic data types operated on directly by machine instructions.

- **Integer / Float**
- **Character / Boolean**

Limited to single scalar values.

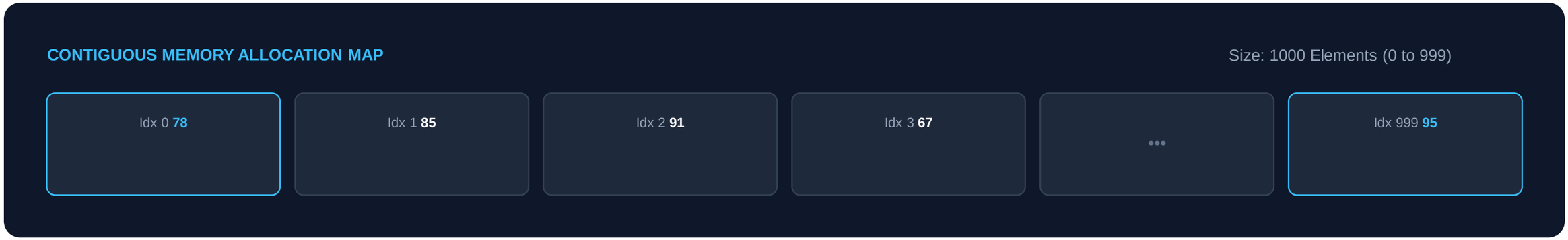
Non-Primitive Structures

Complex data organizations constructed from primitive types.

- **Array**
- Linked List
- Stack
- Queue / Tree

Selected Architecture: One-Dimensional Array

Array Implementation for Student Marks



Sequential Layout

Allocates contiguous bytes in RAM, maximizing CPU cache locality during iteration.

O(1) Direct Access

Calculates base address offset instantaneously: Address = Base + Index * Size.

Problem: Geospatial Route Navigation

Interconnected City Networks

Store spatial topology and travel distances between varying urban endpoints.

Shortest Path Optimization

Calculate optimal transit paths dynamically based on minimal total segment costs.

Non-Linear Topology

Standard linear structures (Arrays, Stacks) cannot model arbitrary cycle networks.



Graph Data Structure Formulation

Vertices (V)

Cities & Intersections

Nodes containing spatial coordinates and location identifiers.

Edges (E)

Road Networks

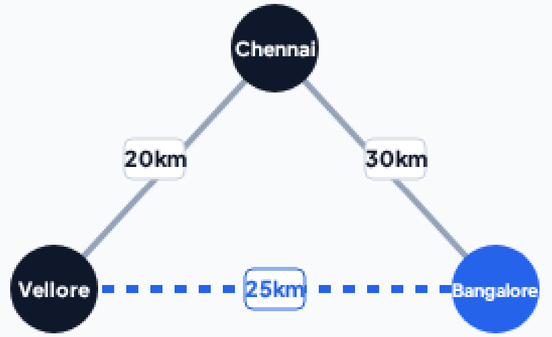
Direct connectivity channels linking adjacent vertices.

Weights (W)

Distance / Cost Metrics

Scalar values representing mileage, traffic delay, or transit expense.

WEIGHTED UNDIRECTED GRAPH MODEL



Algorithmic Route Finding (Dijkstra's)

01

Greedy Initialization

Assigns tentative distances: 0 for source city, infinity for unvisited nodes.

02

Relaxation Step

Evaluates edge weights to adjacent nodes and updates paths if cheaper.

03

Min-Priority Extraction

Picks unvisited node with lowest tentative distance using a Min-Heap.

Time Complexity Analysis

Adjacency List + Min-Heap Execution

O((V + E) log V)

Problem: Backward Page Navigation

State Sequence Tracking

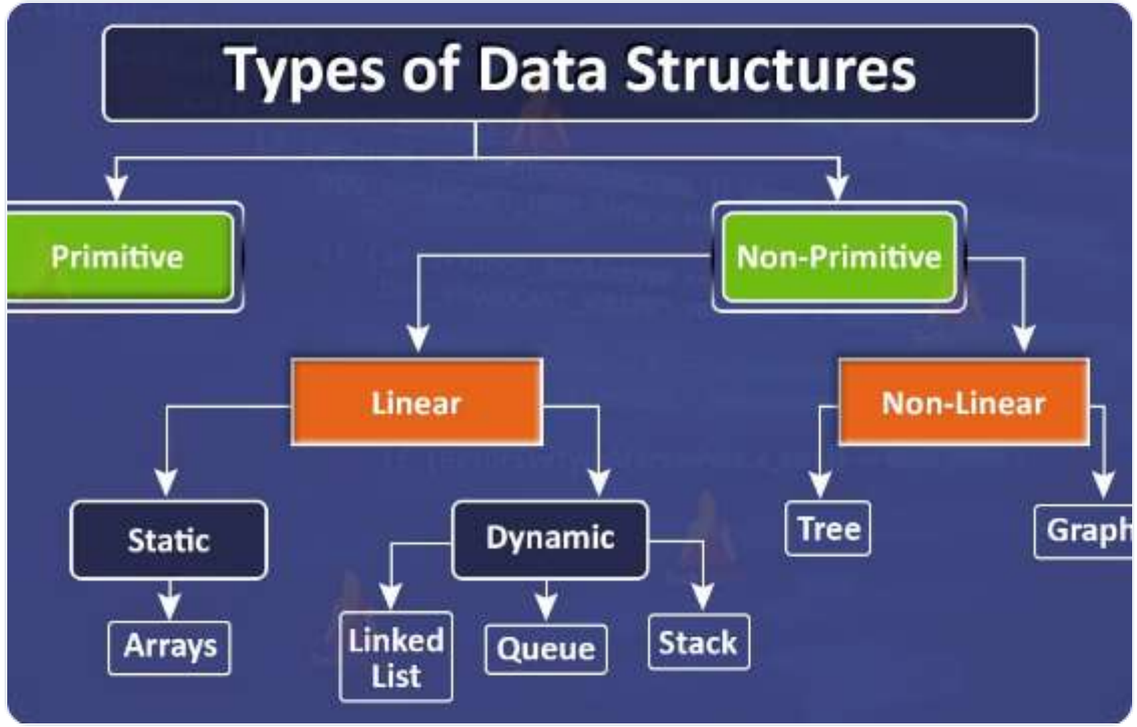
Maintain chronological order of visited web addresses during a browsing session.

Immediate Reversal Access

Pressing "Back" must instantly return to the most recently viewed URL.

Disciplined ADT Choice

Requires a structural policy where item removal strictness matches session behavior.



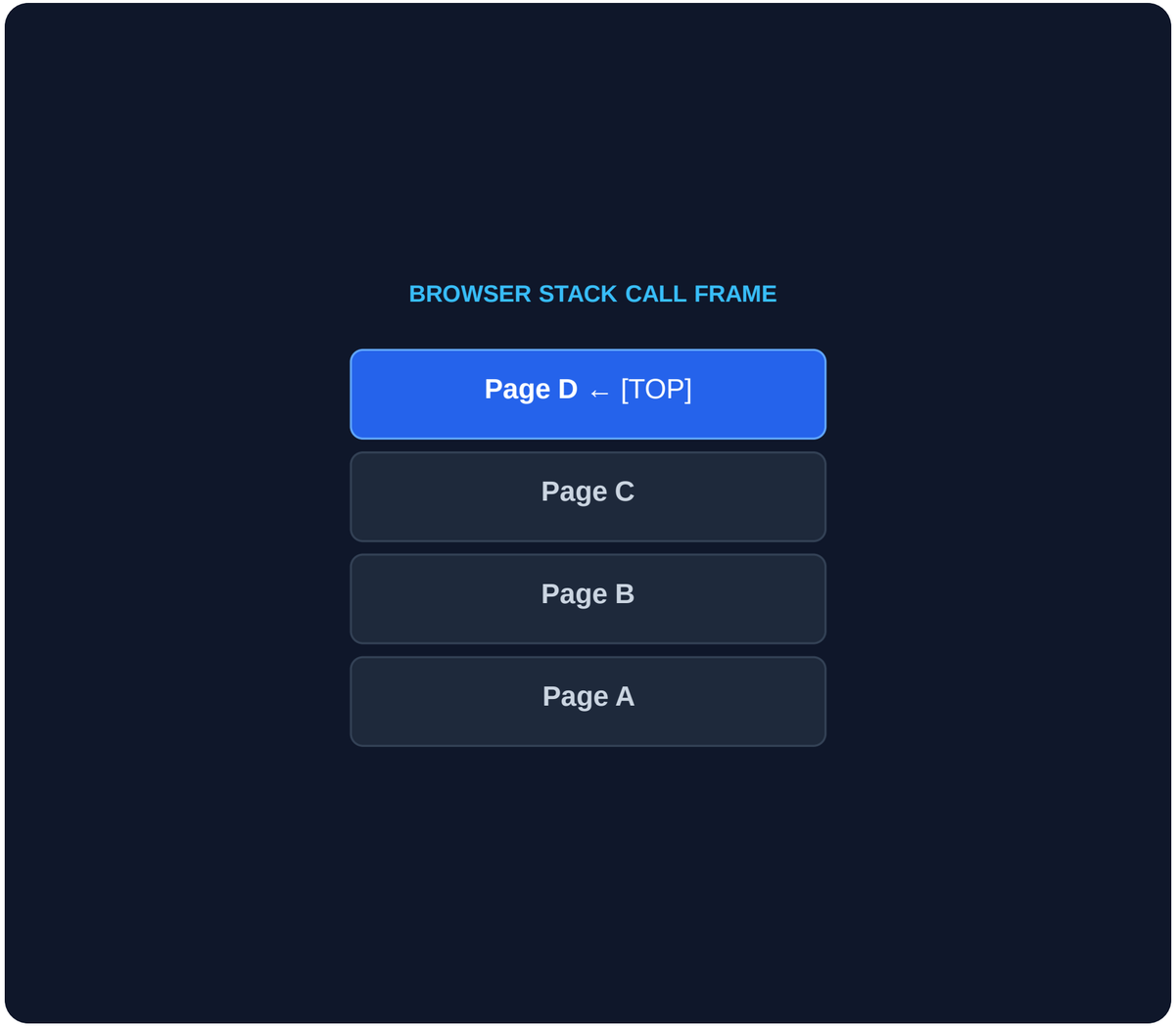
Stack Abstract Data Type (LIFO)

LIFO Operational Rule

Last-In, First-Out: The element inserted last into memory is strictly the first element retrieved.

Single Top Pointer

All insertions and deletions occur exclusively at one designated end (TOP).



Stack Operations & Performance

Operation

Push(URL)

Pushes newly navigated webpage onto the top of the stack.

Time: O(1)

Operation

Pop()

Pops top page when Back is clicked, returning focus to preceding URL.

Time: O(1)

Operation

Peek()

Inspects active page title without modifying stack history state.

Time: O(1)

Trace Example: Google → YouTube → Wikipedia → **College** | [Back Pressed] → Removes **College** → Active: **Wikipedia**

Problem: Massive Book Record Search

50,000 Catalog Records

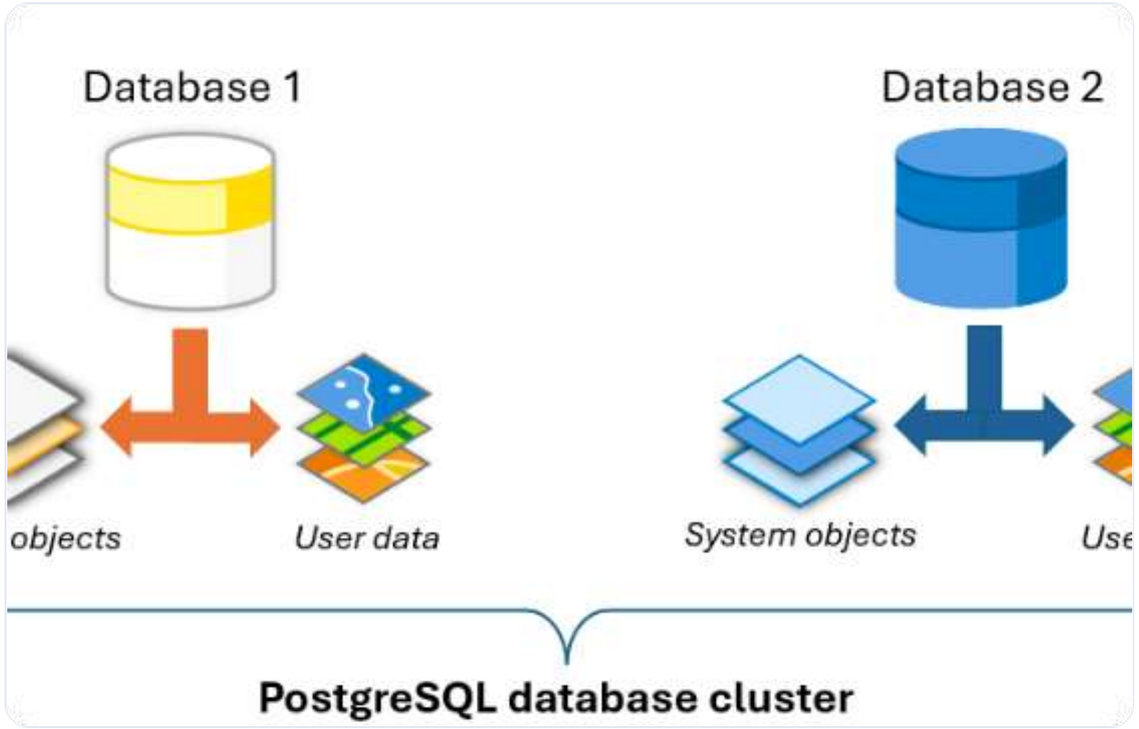
Large dataset containing book titles, availability status, and author details.

ISBN Key Lookup

Searches are conducted using unique 13-digit ISBN alphanumeric keys.

Sub-Second Query Expectation

Linear O(N) traversals create unacceptable system latency at 50,000 items.



Hash Table Mapping Strategy

Key-Value Pair Model

ISBN → Hash(Key) → Index

A mathematical hash function converts arbitrary ISBN strings directly into bounded integer array offsets.

Collision Mitigation

Uses Separate Chaining (Linked Lists) or Open Addressing to resolve index collisions.



Search Complexity Benchmarking (50k Records)

Data Structure Architecture	Average Search Time	Worst Case	Evaluation for 50,000 Records
Unsorted Array	O(N)	O(N)	Infeasible (~50,000 operations/search)
Singly Linked List	O(N)	O(N)	Infeasible (High pointer overhead)
Balanced Binary Search Tree	O(log N)	O(log N)	Moderate (~16 operations/search)
Hash Table	**O(1)**	**O(N)**	**Optimal Choice (Instant 1-step lookup)**

** Note: O(1) average lookup assumes a uniform hash function distribution and a load factor α < 0.75.*





THANK YOU